

Tema 3- Representación del conocimiento: Declarativo y procedimental

Motivación: IA Simbólica vs Subsimbólica

El aprendizaje automático (enfoque subsimbólico) presenta ciertas limitaciones: requiere enormes volúmenes de datos, carece de explicabilidad y tiene serias dificultades para imponer restricciones lógicas en sus soluciones.
La Inteligencia Artificial Simbólica soluciona estas carencias aportando conocimiento explícito, estructura y un verdadero razonamiento simbólico.
La tendencia actual de la industria es la IA híbrida, lo que demuestra que la representación formal del conocimiento sigue siendo esencial.

AGENTES LÓGICOS

Un agente inteligente basado en conocimiento se articula fundamentalmente en dos componentes: una Base de Conocimiento y un Motor de Inferencias

La Base de Conocimiento (KB) es un conjunto de sentencias expresadas en un lenguaje formal que representa aserciones (lo que el agente sabe) sobre el dominio específico.

El Motor de Inferencias está compuesto por algoritmos independientes del dominio encargados de manipular y derivar conocimiento a partir de la KB.

Este diseño corresponde a una aproximación declarativa: construimos el agente simplemente "diciéndole" lo que necesita saber.

El ciclo de funcionamiento del agente utiliza dos operaciones elementales: la función TELL (para incorporar nuevas sentencias originadas a partir de sus percepciones) y la función ASK (para interrogar a la KB sobre qué acción elegir).



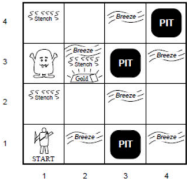
El mundo de Wumpus

El Mundo Wumpus es un entorno simulado para comprobar cómo un agente razona mediante lógica.

Medida de Rendimiento: El agente gana +1000 puntos por salir de la cueva con el oro, pierde -1000 si cae en una fosa o es devorado, y sufre penalizaciones menores (-1) por cada acción o (-10) por usar su única flecha.

Entorno y Sensores: Es una cuadrícula 4x4 donde el agente comienza en . Solo percibe de manera local: detecta Hedor si hay un Wumpus adyacente, Brisa si existe una fosa adyacente, Brillo si hay oro en su casilla, Golpe si choca contra una pared, y un Grito si el Wumpus muere.

Propiedades del entorno: Es un entorno solo parcialmente observable (por depender de la percepción local), pero es determinista, secuencial, estático, discreto y de agente único.



Características

¿Observable?

- No, sólo percepción local

¿Determinista?

- Sí, las salidas están claramente especificadas

¿Episódico?

- No, es secuencial, las acciones no son independientes
- Las recompensas vienen después de ejecutar una o varias acciones

¿Estático?

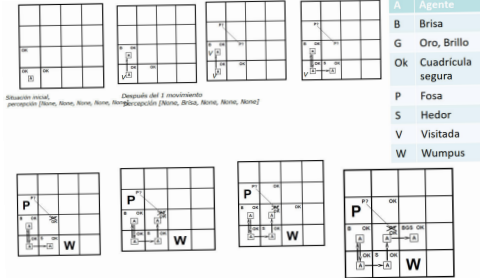
- Sí, el wumpus no se mueve y las fosas tampoco

¿Discreto?

- Sí, todo se puede contar o dividir en partes iguales

¿Agente único?

- Sí, el wumpus, las fosas, los olores, etc. son estáticos y no tienen sus propias metas.



LÓGICA

La lógica es el lenguaje formal que se emplea para representar información, de modo que se puedan inferir conclusiones de forma fiable.

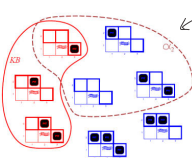
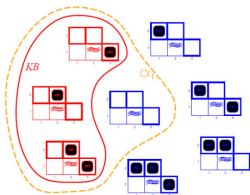
La Sintaxis define estrictamente qué sentencias están permitidas en el lenguaje

- $X+2=Y$ es una sentencia

La Semántica define el "significado" o la verdad de cada sentencia evaluada en los diferentes mundos posibles, formalmente denominados Modelos.

- $X+2=Y$ es cierto en un mundo donde $X=7, Y=9$

Implicación Lógica ($\models$): Decimos que una sentencia α implica lógicamente una sentencia β si y solo si, en absolutamente todos los modelos en los que α es cierta, β también es cierta.



Este no se cumple para todas las bases de conocimiento

KB = REGLAS MUNDO WUMPUS + OBSERVACIONES

$\alpha_1 = "[1,2] \text{ is safe}"$, $KB \models \alpha_1$, probado por model checking (comprobación de modelos)

$\alpha_2 = "[2,2] \text{ is safe}"$, $KB \not\models \alpha_2$

Cualidades de un algoritmo inferencial

- Solidez (Soundness):** El algoritmo es sólido (o preservador de verdad) si toda sentencia que deriva está verdaderamente implicada lógicamente (Analogía: Es como un detector de metales que jamás da falsas alarmas; si pita, seguro hay metal).
- Complejidad (Completeness):** El algoritmo es completo si es capaz de derivar y encontrar cualquier sentencia que esté lógicamente implicada. (Analogía: Es un detector de metales que garantiza encontrar todas las monedas del terreno).

Solidez: i es sólido si

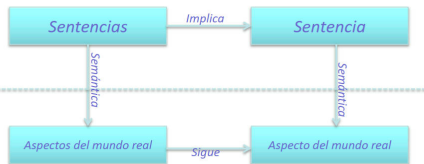
cundo $KB \vdash_i \alpha$ es también cierto que $KB \models \alpha$

Representación

Complejidad: i es completo si

cundo $KB \models \alpha$ es también cierto que $KB \vdash_i \alpha$

Mundo



Monotonización: Estos sistemas lógicos se basan en que, a medida que se añade información a la KB, la cantidad de sentencias inferidas solo puede incrementarse. En la lógica clásica, nunca se invalida una conclusión obtenida previamente (a diferencia del razonamiento humano cotidiano, que puede ser no-monotónico porque a veces "cambiamos de opinión").

LÓGICA PROPOSICIONAL

La lógica proposicional usa símbolos atómicos sencillos unidos mediante conectivas lógicas:

¬ Negación ($\neg$), Conjunción ($\wedge$), Disyunción ($\vee$), Implicación ($\Rightarrow$) y Bicondicional ($\Leftrightarrow$).

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

Inferencia por enumeración

La enumeración en profundidad de todos los modelos es Sólido y Completo

Complejidad temporal: $O(2^n)$ para n símbolos; problema co-NP-completo

Validez: Una sentencia es válida (se la conoce como tautología) si resulta cierta en todos los modelos posibles.

Teorema de Deducción: Indica que la implicación $\alpha \Rightarrow \beta$ se cumple si y solo si, la sentencia implicativa $\alpha \Rightarrow \beta$ es válida.

Satisfacibilidad: Una sentencia es satisfacible si resulta cierta en al menos un modelo.

Inferencia por Reducción al Absurdo: Consiste en deducir que $\alpha \models \beta$ verificando que la conjunción $(\alpha \wedge \neg \beta)$ es lógicamente insatisfacible.

$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$	commutativity of $\wedge$
$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$	commutativity of $\vee$
$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$	associativity of $\wedge$
$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$	associativity of $\vee$
$\neg(\neg \alpha) \equiv \alpha$	double-negation elimination
$(\alpha \Rightarrow \beta) \equiv (\neg \beta \Rightarrow \neg \alpha)$	contraposition
$(\alpha \Rightarrow \beta) \equiv (\neg \alpha \vee \beta)$	implication elimination
$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$	biconditional elimination
$\neg(\alpha \wedge \beta) \equiv (\neg \alpha \vee \neg \beta)$	De Morgan
$\neg(\alpha \vee \beta) \equiv (\neg \alpha \wedge \neg \beta)$	De Morgan
$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$	distributivity of $\wedge$ over $\vee$
$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$	distributivity of $\vee$ over $\wedge$

Tenemos una KB que contiene R1 a R5, y queremos probar $\neg P_{1,2}$.

1. Aplicamos la eliminación incondicional a R2 para obtener:

- R6: $(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$

2. Aplicamos And-Elimination a R6 para obtener:

- R7: $((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$

3. Aplicamos Equivalencia lógica para contrapositivos

- R8: $(\neg B_{1,1} \Rightarrow \neg (P_{1,2} \vee P_{2,1}))$

4. Modus Ponens con R8 y R4

- R9: $\neg (P_{1,2} \vee P_{2,1})$

5. Regla de De Morgan

- R10: $\neg P_{1,2} \wedge \neg P_{2,1}$

• NI (1,2) NI (2,1) CONTIENEN UNA FOSA

R1	$\neg P_{1,1}$
R2	$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
R3	$B_{1,1} \Leftrightarrow (P_{1,1} \vee P_{2,1} \vee P_{3,1})$
R4	$\neg B_{1,1}$
R5	$B_{1,1}$

ESTADO INICIAL	La KB inicial
ACCIONES	Todas las reglas de inferencia aplicadas a todas las sentencias que se emparejen la parte superior de la regla
RESULTADO	Añadir la sentencia en la parte inferior de la regla inferencial
META	Un estado que contiene la sentencia que queremos probar

Usando el mismo proceso que llevó a R10 antes podemos derivar ausencia de fosa en [2,2] y [1,3] (en [1,1] ya sabemos que no hay fosa)

- R13: $\neg P_{2,2}$
- R14: $\neg P_{1,3}$

Podemos aplicar Eliminación bicondicional a R3, y después Modus ponens con R5, para:

- R15: $P_{1,1} \vee P_{2,2} \vee P_{3,1}$

Aplicación de **regla de resolución:**

- Literar $\neg P_{2,2}$ en R13 resuelve con el literal $P_{2,2}$ en R15 para dar el resolvente:
- R16: $P_{1,1} \vee P_{3,1}$
- Literar $\neg P_{1,1}$ en R1 resuelve con el literal $P_{1,1}$ en R16 para dar
- R17: $P_{3,1}$

Prueba por resolución

La **Resolución** es una regla de inferencia fundamental porque proporciona la base para algoritmos inferenciales que son completos. La regla de resolución unidad establece que, si tenemos una disyunción y un literal externo que es el complementario de uno interno, ambos literales se anulan para generar una nueva cláusula resolvente más simplificada.

Forma normal conjuntiva

Para poder usar esta potente técnica, toda sentencia de la lógica proposicional debe traducirse previamente a una **Forma Normal Conjuntiva (CNF)**.

Una sentencia en CNF es básicamente una conjunción principal conectada con ANDs) que contiene múltiples cláusulas, siendo cada cláusula una disyunción (conectada con ORs) de distintos literales.

Teorema Fundamental de la Resolución: Si partimos de un conjunto de cláusulas que resulta ser insatisfacible, el cierre de resolución acabará generando tarde o temprano la "cláusula vacía". (Este es el método de prueba por refutación).

GRAMÁTICA PARA FORMA NORMAL CONJUNTIVA

Sentencia CNF $\rightarrow$ Cláusula₁ $\wedge$ $\wedge$ Cláusula_n

Cláusula $\rightarrow$ Literal₁ $\vee$ $\vee$ Literal_m

Literal $\rightarrow$ Símbolo | $\neg$ Símbolo

FormaCláusulaHorn $\rightarrow$ FormaCláusulaDefinida | FormaCláusulaMeta

FormaCláusulaDefinida $\rightarrow$ (Símbolo₁ $\wedge$ $\wedge$ Símbolo_n) $\Rightarrow$ Símbolo

FormaCláusulaMeta $\rightarrow$ (Símbolo₁ $\wedge$ $\wedge$ Símbolo_n) $\Rightarrow$ Falso

Procedimiento para convertir a CNF

1. Eliminar $\Leftrightarrow$, reemplazar $\alpha \Leftrightarrow \beta$ con $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

2. Eliminar $\rightarrow$, reemplazar $\alpha \Rightarrow \beta$ con $\neg \alpha \vee \beta$

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg (P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

3. Mover $\neg$ hacia dentro usando las reglas de De Morgan y la doble negación.

$$\neg(\neg \alpha) = \alpha \text{ (eliminación doble negación)}$$

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

$$\neg(\alpha \wedge \beta) = (\neg \alpha \vee \neg \beta) \text{ (De Morgan)}$$

$$\neg(\alpha \vee \beta) = (\neg \alpha \wedge \neg \beta) \text{ (De Morgan)}$$

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$$

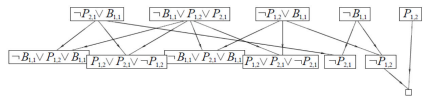
4. La sentencia contiene operadores anidados $\wedge$ y $\vee$ aplicados a literales. Aplicamos la ley distributiva sobre ellos

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

- Ejemplo simple en el mundo wumpus:

- Agente en [1,1], y no hay brisa, así que no puede haber fosas en los cuadrados adyacentes

$$KB = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1} \alpha = \neg P_{1,2}$$



Cláusulas de Horn

Dado que el problema genérico de inferencia (SAT) requiere un tiempo exponencial, se puede restringir la sintaxis a **Cláusulas de Horn** para lograr cálculos eficientes y en tiempo lineal.

Una Cláusula de Horn es una disyunción de literales en la que, como máximo, hay un solo literal positivo.

Cuando la cláusula tiene exactamente un literal positivo, se denomina "cláusula definida" y puede reescribirse intuitivamente como una implicación donde una conjunción de premisas conduce a una conclusión.

$$(\neg L_{u1} \vee \neg B_{u1a} \vee B_{u1}) \text{ se puede escribir } (L_{u1} \wedge B_{u1a}) \Rightarrow B_{u1}$$

Encadenamiento

Bajo esta arquitectura se usan dos grandes motores de razonamiento:

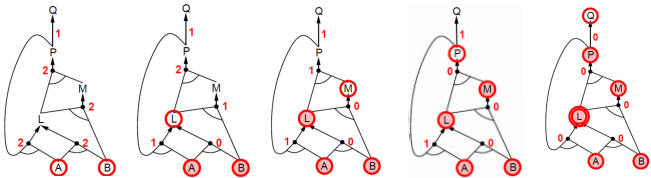
Encadenamiento Progresivo (Forward Chaining)

Es un mecanismo de inferencia que está **dirigido por los datos**.

Empezar desde los hechos concretos de la KB y va disparando de forma progresiva aquellas reglas de las que se cumplen todas sus premisas, añadiendo iterativamente nuevas conclusiones hasta llegar (o no) a nuestra meta.

Es sólido, completo y lineal, pero tiene la desventaja de que puede realizar esfuerzo derivando conclusiones o rutas que son irrelevantes para nuestro objetivo final.

- H1: A
H2: B
R1: $P \Rightarrow Q$
R2: $L \wedge M \Rightarrow P$
R3: $B \wedge L \Rightarrow M$
R4: $A \wedge P \Rightarrow L$
R5: $A \wedge B \Rightarrow L$



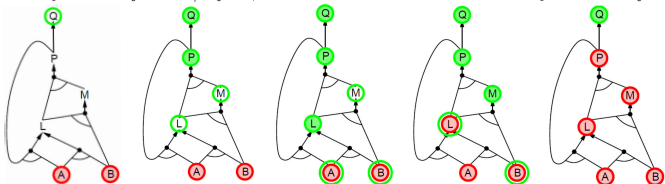
Encadenamiento Regresivo (Backward Chaining)

Es un mecanismo de razonamiento que está **dirigido por las metas**.

Comienza trabajando hacia atrás directamente desde la meta (Query): busca qué reglas de la KB la originan, y luego evalúa si las premisas asociadas a dichas reglas son ciertas, convirtiéndolas en "submetas" temporales.

Su eficiencia suele ser muchísimo mejor (menos que lineal) ya que el proceso se concentra exclusivamente en investigar los hechos que son estrictamente relevantes.

- H1: A
H2: B
R1: $P \Rightarrow Q$
R2: $L \wedge M \Rightarrow P$
R3: $B \wedge L \Rightarrow M$
R4: $A \wedge P \Rightarrow L$
R5: $A \wedge B \Rightarrow L$



Algoritmos de búsqueda local

Muchos de los problemas complejos en ciencias de la computación son problemas de combinatoria que se pueden reducir a buscar la satisfactibilidad proposicional (Problema SAT).

Cuando los espacios de búsqueda son masivos e inabundables, se utilizan algoritmos de búsqueda local, como los conocidos como "Ascensión de Colinas" (HILL-CLIMBING) o "Enfriamiento Simulado" (SIMULATED-ANNEALING).

Un ejemplo característico es el algoritmo probabilístico WALKSAT: este algoritmo asigna valores de verdad al azar y, si falla, escoge de modo aleatorio una cláusula insatisfecha y le cambia un símbolo (flip) con la intención estadística de maximizar el número de cláusulas verdaderas y resolver el problema SAT de modo veloz, aunque incompleto.

```
function WALKSAT (clauses, p, max_flips) returns modelo satisfactorio or fallo.
inputs: clauses, un conjunto de cláusulas en lógica proposicional.
       p, la probabilidad de elegir un "movimiento aleatorio", típicamente sobre 0.5
       max_flips, nº de saltos permitidos antes de rendirse

model ← una asignación aleatoria de true/false a los símbolos en clauses
for i=1 to max_flips do
  if model satisfies clauses then return model
  clause ← una cláusula elegida aleatoriamente de clauses que es falsa en model
  with probability p cambiar el valor en model de un símbolo seleccionado aleatoriamente de clause.
  else cambiar cualquier símbolo en clause que maximice el nº de cláusulas satisfechas

return failure
```

MODELOS DE REPRESENTACIÓN ESTRUCTURADOS

Aunque la lógica de primer orden es muy potente computacionalmente, presenta debilidades en la práctica:

- Escala mal en dominios o bases de conocimiento masivas.
 - Es frágil ante posibles inconsistencias en la entrada de datos.
 - Resulta muy poco intuitiva para los expertos de dominio (médicos, ingenieros) que deben insertar su conocimiento humano en el sistema.
- Los sistemas basados en conocimiento utilizan modelos alternativos estructurados que se clasifican en declarativos y procedimentales

Métodos declarativos

Almacenan verdades estáticas del dominio (hechos). Su gran ventaja es que la información se guarda una sola vez y se puede ampliar la base fácilmente sin modificar lo ya existente.

Destacan dos grandes familias:

Redes semánticas

Representan el conocimiento como un grafo: los nodos son los conceptos/objetos y las aristas (arcos dirigidos) son las relaciones lingüísticas que los unen.

Relaciones típicas (arcos): * Generalización (ES_UN / IS_A): Relaciona una subclase con su superclase, formando taxonomías jerárquicas (Ej. Perra ES_UN Animal).

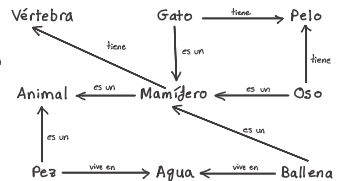
• Agregación (ES PARTE_DE / PART_OF): Relaciona componentes con el todo (Ej. Rueda ES PARTE_DE Coche).

• Ocurrencia (): Relaciona un individuo específico con la categoría genérica a la que pertenece (Ej. "Mili" es un miembro de la clase Perros).

• Relaciones de Propiedad (tiene_pelo, color) y Acción.

Mecanismo de Inferencia Principal: Herencia de Propiedades.

• Cualquier propiedad cierta para una categoría (nodo superior) "cae por gravedad" (se hereda por rastreo de arcos IS_A) a sus subcategorías o ejemplares. Si el nodo "Ave" tiene la relación "Vuela", entonces el nodo "Gorrión" (que ES_UN ave) inferirá automáticamente que vuela, sin necesidad de definirlo explícitamente.



Frames (Marcos)

Es una versión hipervitaminada de las Redes Semánticas. Trata el conocimiento agrupándolo en plantillas estereotipadas.

Tienen una *cabecera* (nombre de la clase, ej. "Paciente") y *Slots* (huecos o atributos como "Edad", "Historial clínico"). Los slots pueden anidarse en varios niveles de profundidad.

Demons: Son el componente procedimental integrado dentro de esta arquitectura declarativa. Son como "Triggers" (disparadores) o manejadores de eventos. Están dormidos hasta que ocurre algo (leer o escribir en un slot específico, if_needed, if_added) y entonces ejecutan rutinas externas o reglas.

(pájaro)	(gorrión)
(morfología)	(morfología)
(plumas)	(plumas)
(pico)	(pico)
(...)	(...)
(reproducción)	(reproducción (ovípara))
(ovípara)	(...)
(...)	(tamaño (pequeño))
(gorrión)	(color (pardo))
(es_un(pájaro))	(...)
(tamaño (pequeño))	
(color (pardo))	

Tipos

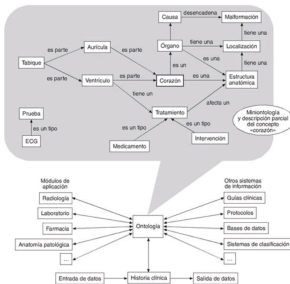
Frames como elementos descriptivos	Frames como elementos de control del conocimiento...
José_Pérez	Hipertensión
Edad	Especificaciones_de_entrada
57_años	Valor_tipo
Historial_clínico	Literal
Hepatitis	Multiplicidad
Tipo	No
B	Incertidumbre
Complicaciones	SI
Fallo_renal	Estrategia
Bronquitis_crónica	Preguntar
Medicación	Usar_reglas
Diuréticos	

Taxonomías, Ontologías y Grafos de conocimiento

Taxonomías: Simples jerarquías de clasificación de conceptos (solo admiten relación IS-A).

Ontologías: Extienden a las taxonomías porque dotan de significado riguroso y explícito al vocabulario. Fijan los conceptos, las relaciones admitidas, las propiedades y qué restricciones se aplican (definiendo un 'Vocabulario Compartido').

Las Ontologías son la capa de base de metadatos o esquema con los que luego se construyen **Grafos de Conocimiento** enormes, que sirven para buscar, integrar bases de datos complejas e interoperar y son la base de los modernos motores de búsqueda y de integración con IA subsimbólica.



Interoperabilidad
Razonamiento automático
Explicabilidad
Consistencia del conocimiento
Conexión con otras aplicaciones

Modelos Procedimentales: Reglas de Producción

Representan un carácter mucho más dinámico y heurístico. Es la clásica estructura algorítmica

IF... THEN... ELSE.

Ventajas: Máxima explicabilidad (el sistema te puede decir exactamente qué hilo de reglas conectadas se usó para deducir un resultado) y alta modularidad (cada regla es una unidad de conocimiento por sí sola).
Desventajas: El mantenimiento es muy complejo y escalan mal en bases del conocimiento con cientos de miles de reglas por la dificultad de gestión de conflictos.

Tipos de evaluación en el antecedente (Premisa):

- **IFALL:** Es necesario que todas las cláusulas conectadas por un AND lógico sean ciertas.
- **IFANY:** Cláusulas separadas por OR lógico; la regla se dispara en cuanto se detecta la primera cláusula que cumple, abortando de forma no exhaustiva la investigación del resto.
- **IFSUME:** Igual al OR anterior; pero obliga a hacer una comprobación exhaustiva del resto de cláusulas del antecedente antes de saltar

Pronóstico

- Meteorológico
- Buen tiempo
- Enturbiamiento
- Empeoramiento
- Otros
- otros

(pronóstico)

(meteorológico)

(buen tiempo)

(enturbiamiento)

(empeoramiento)

(otros))

(otros))

Actividades

- Recomendadas
- Al aire libre
- Quedarse en casa
- otras

(actividades)

(recomendadas)

(al aire libre)

(quedarse en casa))

(otras))

Regla_3

IF: (1) (pronóstico meteorológico) = enturbiamiento

and: (2) (cielo nubes) = presentes

and: (3) (cielo nubes presentes posición) = capas_bajas

THEN: (pronóstico meteorológico empeoramiento)

Regla_4

IF: (1) (pronóstico meteorológico) = buen tiempo

THEN: (actividades recomendadas al aire libre)

ELSE: (actividades recomendadas quedarse en casa)

SISTEMAS DE PRODUCCIÓN

Arquitectura de Sistemas de Producción

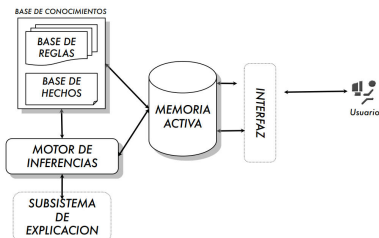
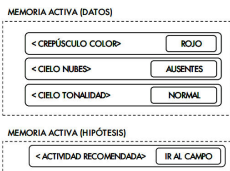
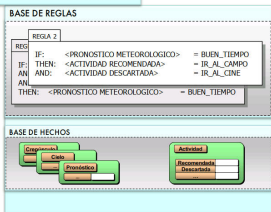
Base de Conocimiento (KB): Describe el dominio del problema. Está dividida en dos esqueletos que deben ser estrictamente compatibles entre sí:

• **Base de Hechos (BH):** Es el esqueleto declarativo. Almacena todas las verdades estáticas, características y parámetros relevantes del dominio.

• **Base de Reglas (BR):** Es el esqueleto procedimental. Contiene las reglas lógicas (con formato IF <premisas> THEN <conclusiones>) que permiten construir el razonamiento. Memoria Activa (MA): contiene la información estática de un problema, almacena los cambios de estado (estado actual) e interactúa con el exterior. Es el "foco de atención" del sistema y contiene:

- Datos iniciales e introducidos posteriormente
- Hechos inferidos
- Hipótesis o metas aún no obtenidas

BASE DE CONOCIMIENTOS



Motor de Inferencias (MI): Actúa como el intérprete y el controlador. Separa radicalmente el mecanismo de control de la base de conocimiento subyacente.

Subsistema de Explicación: Encargado de dotar de trazabilidad a las decisiones.

Ciclo básico del motor de inferencias (Sistema de Producción)

El Motor de Inferencias evalúa y procesa la información mediante un bucle constante llamado ciclo básico, dividido en dos grandes fases:

Fase 1: Decisión (Selección de reglas)

Su objetivo es decidir qué regla exacta de la Base de Reglas debe dispararse. Se realiza en 3 pasos:

1. **Restricción:** Para no ralentizar el sistema, se filtran macro-grupos de reglas que no tienen nada que ver con el contexto actual de la Memoria Activa (suele usar metaconocimiento).

2. **Equiparación (Emparejamiento):** Se verifica cuáles de las reglas restantes encajan lógicamente con los datos de la Memoria Activa. Todas las reglas válidas encontradas pasan a conformar lo que se llama el "Conjunto Conflicto".

3. **Resolución de Conflictos:** Si el conjunto conflicto tiene varias reglas disponibles, el MI debe elegir solo una. Para ello usa criterios como:

- Elegir la regla con mayor prioridad programada.
- Elegir la regla más específica (la que evalúe más condiciones en su IF, porque es más detallada).
- Elegir la regla que utilice la información o datos más recientes (para mantener una línea de razonamiento coherente).
- Usar metarreglas (ej. "Priorizar siempre reglas de seguridad médica frente a diagnósticos generales").

Fase 2: Acción (Ejecución)

1. Se dispara la regla elegida en la resolución de conflictos.

2. Se actualiza la **Memoria Activa** introduciendo las nuevas conclusiones deducidas.

3. Se marca la regla como ejecutada para no crear bucles infinitos y se evalúa si debe iniciar un nuevo ciclo o si ya se ha llegado al final (la meta se alcanza).

$BR = [R1, R2, R3, R4, R5, R6, R7]$

R1:	IF (X and Y)	THEN	Z
R2:	IF (C and D)	THEN	G
R3:	IF (E and V)	THEN	H
R4:	IF (A and B)	THEN	C
R5:	IF (F or G)	THEN	X
R6:	IF (Z and B)	THEN	V
R7:	IF (E and C)	THEN	F

$MO = \langle A, B, D, Y, E, (H) \rangle$

Datos del problema o hechos previamente demostrados

Hipótesis sin confirmar

Dinámicas de tránsito: Progresivo vs Regresivo

En función de cómo esté configurado el Motor de Inferencias, el razonamiento fluirá de manera muy distinta:

Encadenamiento Progresivo (Dirigido por los Datos)

• Funcionamiento: Analiza los antecedentes (IF) de las reglas. Si las premisas coinciden con los hechos de la Memoria Activa, la regla se ejecuta y genera un nuevo hecho (el THEN). Este proceso empuja información siempre hacia adelante.

• Ventajas / Desventajas: Es computacionalmente rápido (lineal respecto a la KB) pero puede llegar a procesar ramas enteras y deducciones que no son relevantes para el objetivo que en el fondo buscamos.

Encadenamiento Regresivo (Dirigido por los Objetivos o Metas)

• Funcionamiento: El sistema comienza con una meta específica en la Memoria Activa. El MI rastrea las conclusiones (THEN) de las reglas para ver cuáles darían como resultado esa meta. Las premisas (IF) de esas reglas se extraen y se añaden a la Memoria Activa como submetas (hipótesis temporales).

• Este proceso (llamado Retropropagación) continúa iterativamente hasta que las submetas requeridas emparejan mágicamente con datos puros y reales que el sistema ya tenía. Cuando esto ocurre, se produce la fase de Ejecución, resolviendo toda la cadena hacia la meta.

• Ventajas: Es excelente para la resolución de problemas enfocados y el diagnóstico, porque el sistema ignora toda la base de conocimiento que no está relacionada con la meta inicial.

Ejemplo

Encadenamiento progresivo

$M_0 = \langle A, B, D, Y, E, (H) \rangle$ $H_0 \rightarrow N_0$

Paso 1) $RA = R_4$ $RE = R_4$ $M_1 = \langle A, B, D, Y, E, (H), C \rangle$ H_1

Paso 2) $RA = R_7, R_2$ $RE = R_2$ $M_2 = \langle A, B, D, Y, E, (H), C, G \rangle$ H_2

Paso 3) $RA = R_7, R_5$ $RE = R_5$ $M_3 = \langle A, B, D, Y, E, (H), C, G, X \rangle$ H_3

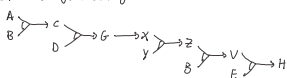
Ejecuto la más reciente, sigo profundizando

Paso 4) $RA = R_7, R_1$ $RE = R_1$ $M_4 = \langle A, B, D, Y, E, (H), C, G, X, Z \rangle$ H_4

Paso 5) $RA = R_7, R_6$ $RE = R_6$ $M_5 = \langle A, B, D, Y, E, (H), C, G, X, Z, V \rangle$ H_5

Paso 6) $RA = R_7, R_3$ $RE = R_3$ $M_6 = \langle A, B, D, Y, E, C, G, X, Z, V, H \rangle$ $H \rightarrow STOP$

Circuito inferencial generado:



Encadenamiento regresivo

$M_0 = \langle A, B, D, Y, E, (H) \rangle$ $H_0 \rightarrow N_0$

Paso 1) $RA = R_3$ $RA: RE = -$ $M_1 = \langle A, B, D, Y, E, (H), (V) \rangle$ H_1

Paso 2) $RV = R_6$ $RA: RE = -$ $M_2 = \langle A, B, D, Y, E, (H), (V), (Z) \rangle$ H_2

Paso 3) $RV = R_1$ $RA: RE = -$ $M_3 = \langle A, B, D, Y, E, (H), (V), (Z), (X) \rangle$ H_3

Paso 4) $RV = R_5$ $RA: RE = -$ $M_4 = \langle A, B, D, Y, E, (H), (V), (Z), (X), (G) \rangle$ H_4

Paso 5) $RV = R_7, R_2$ $RA: RE = -$ $M_5 = \langle A, B, D, Y, E, (H), (V), (Z), (X), (G), (C) \rangle$ H_5

Me quedo ambos

Paso 6) $RV = R_4$ $RA: R_4 = RE$ $M_6 = \langle A, B, D, Y, E, (H), (V), (Z), (X), (G), (C) \rangle$ H_6

Paso 7) $RA = R_2, R_3 = RE$ $M_7 = \langle A, B, D, Y, E, (H), (V), (Z), (X), (G), (C) \rangle$ H_7

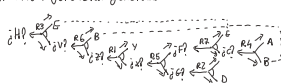
Paso 8) $RA = R_5 = RE$ $M_8 = \langle A, B, D, Y, E, (H), (V), (Z), (X), (G), (C), F, X, Z \rangle$ H_8

Paso 9) $RA = R_1 = RE$ $M_9 = \langle A, B, D, Y, E, (H), (V), (Z), (X), (G), (C), F, X, Z, V \rangle$ H_9

Paso 10) $RA = R_6 = RE$ $M_{10} = \langle A, B, D, Y, E, (H), (V), (Z), (X), (G), (C), F, X, Z, V \rangle$ H_{10}

Paso 11) $RA = R_3 = RE$ $M_{11} = \langle A, B, D, Y, E, C, F, G, X, Z, V, H \rangle$ $H \rightarrow STOP$

Circuito inferencial generado:



Proceso evolutivo: retropropagación

Cadena inferencial: quevucosn